

Phase Template

Document: docs/architecture/PHASE_TEMPLATE.md

System: HackNest — Student Operating System

Status: Foundational / Authoritative — closes a previously-missing Execution-layer document

Type: Template Specification Only — No Implementation

Follows README.md and MASTER_ARCHITECTURE.md exactly. Per MASTER_ARCHITECTURE.md Section 9, this document is the sole authoritative owner of the structure every phase in IMPLEMENTATION_ROADMAP.md must follow. IMPLEMENTATION_ROADMAP.md invented its own phase structure in this document's absence; Section 3 below reconciles that structure against the canonical one defined here.

1. Purpose

A roadmap is only as trustworthy as the consistency of its phases. If one phase defines "done" one way and another phase defines it a different way,

dependency analysis (`MASTER_ARCHITECTURE.md` Section 6) and audit (`ARCHITECTURE_VALIDATION_REPORT.md` -style review) become unreliable. This document fixes one canonical shape every phase must use, so that any two phases can be compared, audited, and sequenced on equal footing.

2. Canonical Phase Fields

Every phase in `IMPLEMENTATION_ROADMAP.md` (or any future roadmap) must contain exactly these fields, in this order:

1. **Phase Number** — a fixed, sequential, three-digit identifier (e.g., `007`), never reused or reordered once assigned.
2. **Phase Name** — a short, descriptive name matching the domain(s) it primarily implements, per `DATABASE_ARCHITECTURE.md` Section 4's naming.
3. **Objective** — one to two sentences stating what capability exists once this phase is done.
4. **Scope** — an explicit boundary statement: what this phase *does* and, where ambiguity is likely, what it explicitly *does not* cover (e.g., "implements team creation and membership; does not implement team discovery, which is a separate

phase"). This field was missing from the roadmap's original structure and is added per this document's own audit finding.

5. **Prerequisites** — which earlier phases must already be complete before this phase may begin.
 6. **Deliverables** — the concrete capabilities/artifacts produced.
 7. **Dependencies** — which other phases this phase relies on architecturally, distinguishing **hard dependencies** (cannot function at all without) from **soft dependencies** (functions in reduced form without), per `MASTER_ARCHITECTURE.md` Section 6's requirement that dependency kind be stated explicitly.
 8. **Estimated Complexity** — Low / Medium / High, a relative sizing signal, not a time estimate.
 9. **Acceptance Criteria** — specific, checkable statements of what must be true for the phase to be considered functionally complete.
 10. **Validation Checklist** — specific checks tying back to `TESTING_STRATEGY.md`'s testing levels and `SECURITY_ARCHITECTURE.md`'s Security Review Checklist where applicable.
 11. **Expected Output** — a one-sentence summary of the tangible end state.
-

3. Reconciliation Against

`IMPLEMENTATION_ROADMAP.md`'s Existing

Structure

The roadmap's original, self-invented structure (Phase Number, Phase Name, Objective, Prerequisites, Deliverables, Dependencies, Estimated Complexity, Acceptance Criteria, Validation Checklist, Expected Output) matches this canonical template with **one addition**: the **Scope** field (position 4 above) did not previously exist as a distinct field — the roadmap folded scope into Objective and Deliverables together.

This is confirmed as the only structural gap between the roadmap's existing phases and this canonical template. All 27 existing phases otherwise already conform. Bringing every phase fully into conformance (adding an explicit Scope line to each) is a mechanical, low-risk follow-up edit to `IMPLEMENTATION_ROADMAP.md`, tracked as an open action rather than performed silently by this document (which does not modify the roadmap itself).

4. Rules for Applying This Template

- No phase may omit a field; if a field is genuinely not applicable (rare), it must say so explicitly rather than being silently absent.
 - **Dependencies must be acyclic**, per `MASTER_ARCHITECTURE.md` Section 6 — a phase may never list, directly or transitively, a dependency on a phase that depends on it. Any new phase proposal that would introduce a cycle is rejected at proposal time under the Section 3 amendment process in `MASTER_GOVERNANCE_AMENDMENT_001.md`.
 - **Acceptance Criteria must be independently checkable** — not "works well," but a specific, falsifiable statement.
 - **Validation Checklist items must cite the owning subsystem document** (e.g., "per `TESTING_STRATEGY.md`'s Authorization Testing") rather than inventing ad hoc validation criteria per phase.
-

Definition of Done

- [x] Template specification only — no implementation.
- [x] Follows `README.md` and `MASTER_ARCHITECTURE.md` exactly.

- [x] Reconciled explicitly against `IMPLEMENTATION_ROADMAP.md`'s existing structure, with the one gap (Scope) identified rather than glossed over.
-

Validation Report

Internal consistency: the 11-field template is self-consistent and each field's purpose is distinct (no two fields overlap in what they capture). **Consistency with governing documents:** aligns with

`MASTER_ARCHITECTURE.md` Section 6's dependency-kind requirement and Section 8's naming conventions.

Outstanding follow-up: `IMPLEMENTATION_ROADMAP.md` should be revised to add the Scope field to all 27 phases; this is tracked as an open, low-risk action rather than resolved here.

Status: Validated — Execution Layer Template Complete